Spis treści

FunctionSolver 2

Wstęp 3

Funkcja 4

Calc() 4

LinearFunction 5

LinearFunction() 5

Calc() 5

IsPerpendicularTo() 5

angle() 6

QuadraticFunction 7

QuadraticFunction() 7

Calc() 7

Vertex() 7

Intersection() 7

Opis testów 8

Art Gallery 10

Specyfikacja techniczna projektu 11

Opis projektu 11

Komponenty aplikacji 11

Opis testów 12

PrimeSeek 13

Opis 14

Opis flag 14

Opis algorytmu sita 14

Specyfikacja 15

Funkcje 15

firstOfMult 15

sieveSegment 16

customSieveSegment() 16

convertToDurationString 16

convertToDurationString(u64 duration) 17

Testy 18

Benchmark 19

Wymagania aplikacji 20

Definicje Testów 21

Testy jednostkowe 21

Testy integracyjne 21

Testy systemowe 21

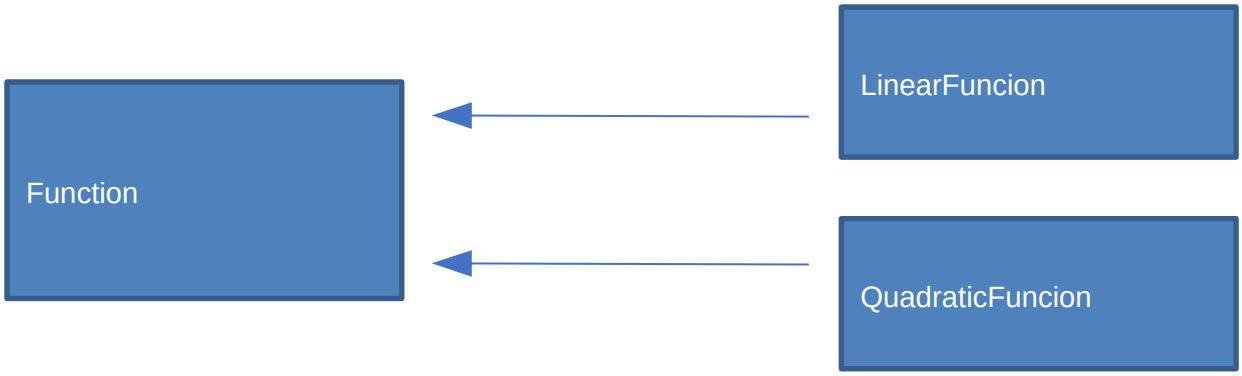
Testy akceptacyjne 21

FunctionSolver

Wstęp

Program służy do obliczania punktów wspólnych funkcji kwadratowej i liniowej. Tworzę 2 klasy potomne, które przez konstruktory przyjmują współczynniki do samych siebie.

Działają na liczbach zmiennoprzecinkowych pojedynczej precyzji (float'y) w celu szybszego obliczania wartości. Więcej liczb przy tym można upakować do rejestrów SIMD (jak XMM, YMM, ZMM), co znacznie przyspiesza obliczenia.



Funkcja

abstract class # Funkcja

Klasa bazowa dla dowolnej funkcji.

Metody

virtual float ## Calc(float x)

Oblicza wartość funkcji

Argumenty:

- x (float) - argument

Wartość zwracana: Wynik wyrażony w typie FP32

LinearFunction

class LinearFunction

Klasa reprezentująca funkcję liniową w postaci kierunkowej. Można obliczyć poza punktami wspólnymi również kąt między nią a osią OX, czy prostopadłość między dwoma funkcjami liniowymi

Metody

LinearFunction(float a, float b)

Inicjalizacja funkcji liniowej

Argumenty:

- a, b (float) - współczynniki

Ograniczenia:

Wartość zwracana: Obiekt typu *LinearFunction*

float Calc(float x)

Oblicza wartość funkcji

Argumenty:

- x (float) - argument

Wartość zwracana: Wynik wyrażony w typie FP32

bool isPerpendicularTo(*LinearFunction* _f2)

Sprawdza prostopadłość dwóch funkcji liniowych (prostych). Korzysta z zależności:

Argumenty:

- _f2 (LinearFunction) - inna funkcja liniowa

Wartość zwracana: *true* dla prostopadłych, *false* dla nieprostopadłych

`float` angle(bool returnInRadians = false)

Oblicza kąt między osią OX, a samą funkcją (inaczej argument). Kąt jest wartością z przedziału [0˚, 360˚] lub [0, 2π] przy zwracaniu w radianach

Argumenty:

- returnInRadians (bool) – czy zwrócić w radianach

Wartość zwracana: domyślnie wartość zwracana w stopniach, przy ustawieniu argumentu funkcji na *true* w radianach

`(float?, float?) / float?` Intersection(<Klasa > _funcB)

Oblicza punkty wspólne funkcji z inną funkcją.

Argumenty:

- _funcB (LinearFunction/QuadraticFunction) – druga funkcja

Wartość zwracana:

Kolejno:

- (null, null) / null – dla braku rozwiązań z funkcją kwadratową / liniową
- (float, null) / float – dla jednego rozwiązania z funkcją kwadratową / liniową
- (float, float) – dla dwóch rozwiązań z funkcją kwadratową

QuadraticFunction

class QuadraticFunction

Klasa reprezentująca funkcję kwadratową w postaci ogólnej. Automatycznie przy inicjalizacji oblicza wierzchołek funkcji

Metody

QuadraticFunction(float a, float b, float c)

Inicjalizacja funkcji liniowej

Argumenty:

- a, b, c (float) - współczynniki

Ograniczenia:

Wartość zwracana: Obiekt typu *QuadraticFunction*

float Calc(float x)

Oblicza wartość funkcji

Argumenty:

- x (float) - argument

Wartość zwracana: Wynik wyrażony w typie FP32

(float, float) Vertex()

Zwraca współrzędne wierzchołka paraboli

Wartość zwracana: Krotka 2-wartościowa oznaczająca kolejno argument i wartość funkcji przy niej (p i q)

float?, float?) / float? Intersection(<Klasa > _funcB)

Oblicza punkty wspólne funkcji z inną funkcją.

Jeśli chodzi o sposób obliczania punktów wspólnych, jest dokonane po prostu przez przyrównanie dwóch równań parabol.

Dla uproszczenia:

Wzór na wyróżnik funkcji kwadratowej wtedy będzie wyglądał tak:

Zatem:

A wiedząc że liczba rozwiązań jest skorelowana z wartością wyróżnika, więc wartością zwracaną będzie tutaj jedna z 3 możliwości:

- (null, null),
- (float, null),
- (float, float),

Podobnie w taki sposób można wyszukać punktów wspólnych z funkcją liniową.

Argumenty:

- _funcB (LinearFunction/QuadraticFunction) – druga funkcja

Wartość zwracana:

Kolejno:

- (null, null) / null – dla braku rozwiązań z funkcją kwadratową / liniową
- (float, null) / float – dla jednego rozwiązania z funkcją kwadratową / liniową
- (float, float) – dla dwóch rozwiązań z funkcją kwadratową

Opis testów

Nazwa	Opis	SWT	Warunek
zerowaFunkcjaLiniowa	Czy jednocześnie a i b obiektu są zerowe?	A	a ≠ 0 && b ≠ 0
wynikNiewymierny	Czy funkcja potrafi dobrze zwrócić wartość, która	E	

	będzie odpowiednio precyzyjna?		
RozwiazaniaFnKwadratowej	Czy biblioteka zwróci poprawne rozwiązania punktów wspólnych funkcji liniowej i kwadratowej?	F	
najwiekszyWspolRownyZero	Czy współczynnik A fn. kwadratowej jest równy zero?	C	
zwrocPoprawykat	Zwróć kąt z naciskiem na dokładność, jaką może zaoferować typ float	F	
nieProstopadle	Czy proste są prostopadłe?	F	$A_k \times A_l \neq -1$
Prostopadle	Czy proste nie są prostopadłe?	F	$A_k \times A_l = -1$

Art Gallery

Specyfikacja techniczna projektu

Typ aplikacji	webowa
Zastosowany framework frontendowy	React.js
Pozostałe technologie	HTML 5, CSS 3, Babel (do Jest'a - obsługa .test.tsx)
Język programowania	TypeScript
Framework do testowania	Jest

Opis projektu

ArtGalleryReact to prosta aplikacja webowa, która wyświetla obraz oraz dane o nim, jak data wystawienia, tytuł. krótki opis. Dane pochodzą od otwartoźródłowego API stworzonego na *openapi.json.* Dane pochodzą od *Art Institute of Chicago* (dokumentacja API, strona główna). Można wylosować dzieło sztuki, poznać podstawowe informacje o obrazie.

Zdjęciowe API *Art Institute of Chicago* jest zgodny ze standardem *IIIF 2.0*.

Grafiki zostały obraobione w taki sposób, by mogły pasować do stylu aplikacji, grafiki SVG zostały wzięte ze strony SVGRepo

Komponenty aplikacji

- SideBar - pasek boczny, można go wyłączyć przyciskiem ze strzałką. W nim znajduje się również logo *Art Institute of Chicago*, oraz przycisk do losowania kolejnej pracy artystycznej;
- *ImageButton* – komponent do przycisku z obrazkiem, przyjmuje 2 argumenty:
  - obrazek (jako obiekt uprzednio zaimportowany w taki sposób: *import logo from "<url do obrazka>"*),
  - funkcję strzałkową przypisującą obiektowi przy zdarzeniu naciśnięcia przycisku;

- Główny (*App*) – odpowiada za:
  - wysyłanie requesta przy starcie przy każdej zmianie ID entry
  - wyświetlenie danych

Opis testów

Nazwa	Opis	SWT[1]	Warunek
isRenderingProperly	Sprawdza, czy przy inicjalizacji komponentu, aplikacja nie wyrzuca błędów I poprawnie wyświetla co najmniej tytuł I rok utworzenia	F	strlen(title) > 0 && strlen(date) > 0
GoodOutputFromRandomization	Przy losowaniu id kolejnego dzieła sztuki, aplikacja renderuje jak powinna, a nie pozostawia sam czarny ekran	E	*Tak jak opisano*
IsImageVisible	To jest szczególny przypadek testowy, ponieważ API mają w zwyczaju osłaniać się zabezpieczeniami od CloudFlare I próba pobrania go, nawet poprzez fetch API, wyrzuca response HTTP 403 (*Forbidden*) Ten test sprawdzi szczególnie, czy aplikacja może się oprzeć zabezpieczeniom.	F	Zdjęcie się wyświetla bez problemu
IsSideBarHidingProperly	Czy po naciśnięciu przycisku pasek boczny chowa się prawidłowo?	B	Pasek boczny chowa się prawidłowo

1 Skala Ważności Testu (SWT):
A - Opcjonalny
B – Prawie ważny
C – Ważny
D – Bardzo ważny
E – Prawie krytyczny
F - Krytyczny


PrimeSeek

Opis

Primes to aplikacja do liczenia funkcji π(x) – funkcji liczącej ilość liczb pierwszych od 2 do x, oraz do generowania liczb pierwszych do pliku

Może wypisać do pliku liczby te w postaci binarnej (w formacie big-endian, każda liczba 64-bitowa).

Program używa znanego sita Erastonetesa, w wersji segmentowanej, z modyfikacją taką, że sito nie zawiera liczb parzystych (oprócz 2 wszystkie liczby naturalne parzyste są złożone), co redukuje o połowę wykorzystanie pamięci, ale narzuca na barki procesora obliczenia związane z liczeniem pozycji początkowej kursora.

Złożoność obliczeniowa sita Erastonetesa wynosi $O(n*loglog(n))$.

Obsługuje zewnętrzne pliki z listą liczb pierwszych oddzielonymi spacją (tzw. słowniki liczb pierwszych - prime_dict)

Program liczy czas wyznaczania liczb pierwszych.

Program również obsługuje wielowątkowość. Dlatego sito jest segmentowane. Pewne zmienne zostały ograniczone mutex'ami (jak bufor liczb pierwszych, zmienna licząca ogólną liczbę znalezionych liczb pierwszych), by wątki nie mogły mieć dostępu do nich jednocześnie. Każdy rdzeń wykonuje określony zakres liczb. Rozmiar każdego fragmentu to 100 000, ale tą wartość da się zmienić przed kompilacją.

Opis flag

ReturnPrimes	Wypisuje liczby do plików
FromScratch	Nie używa zewnętrznego słownika, a oblicza od początku. Ustawienie flagi na *false* sprawi, że program odczyta słownik, i będzie wykreślał liczby według liczb w słowniku
WithVerification	Dodatkowo weryfikuje każdą wyliczoną liczbę, czy jest pierwsza, najprostszą metodą.
BinaryOutput	Wypisuje liczby w postaci binarnej, zamiast liczbowej – głównie w celu oszczędności danych, gdyż nawet 19-cyfrowa liczba może zostać zapisana na 8 bitach, zamiast na 19.
Sieving	Użyj sitowania, nie metody naiwnej

Opis algorytmu sita

Specyfikacja

Typ projektu	Aplikacja konsolowa / Obliczenia naukowe
Język programowania	C++11
Framework do testowania	GoogleTest (gtest)
Dodatkowe technologie	AddressSanitizer

Funkcje

inline u64

findIndexOfFirstFullyDivisable(u64 num, u64 mul)

Sito jest zmodyfikowane tak, że zawiera tylko liczby nieparzyste, oraz w celu wsparcia wielowątkowości. Może się zdarzyć, że sito będzie przetwarzać różne zakresy, więc należy wyznaczyć indeks pierwszej liczby, od której następne liczby będą wykreślane.

Algorytm wyznaczania tego indeksu wygląda następująco:

K1: Jeśli modulo z dzielnika jest równe 0, **zwróć indeks 0**
K2: Wylicz *odwrotność* modułu, tj. dzielnik - (num % dzielnik)
K3: Stwórz tymczasową zmienną _num, która będzie przechowywał tymczasowy stan *num*
K4: Dodaj odwrotność modułu do zmiennej tymczasowej *_num*
K5: Jeżeli *_num* po dodaniu odwr. Modułu będzie parzysta dodaj wartość dzielnika. (Wykluczyłem przecież wszystkie liczby parzyste)
K6: Końcowy indeks to : (_num - num) / 2

Przykład:

num := 69, mul := 17
69 % 17 = 1
odwMod := 16
_num := num
_num + odwMod
_num = 85

ret (85 - 69) / 2
ret 7

Argumenty:

- num - liczba całkowita dodatnia
- mul - dzielnik

Wartość zwracana: index pierwszej liczby w sicie, który jest podzielny przez *mul*

u64 sieveSegment(u64 start)

Wrapper dla wątków do podsit o stałym rozmiarze. Używa std::bitset.

Argumenty:

- start – wartość początkowa
- chunkSize – rozmiar zakresu do sita (sito jest o połowę mniejsze od samego zakresu – nie zawiera liczb parzystych)

Ograniczenia:

$$start > 0 \land chunkSize > 0 \left(ograniczone\, typowo \right)$$

Wartość zwracana: liczba znalezionych liczb pierwszych w segmencie

u64 customSieveSegment(u64 start, u64 chunkSize)

Wrapper dla wątków do podsit o dowolnym rozmiarze. Używa zwykłej tablicy boolean'ów, zamiast std::bitset

Argumenty:

- start – wartość początkowa
- chunkSize – rozmiar zakresu do sita (sito jest o połowę mniejsze od samego zakresu – nie zawiera liczb parzystych)

Ograniczenia:

$$start > 0 \land chunkSize > 0 \left(ograniczone\, typowo \right)$$

Wartość zwracana: liczba znalezionych liczb pierwszych w segmencie

void convertToDurationString(u64 duration)

Specjalistyczna funkcja, która wyświetla czas podany w mikrosekundach na łatwy do zrozumienia format.

Liczba mikrosekund	Wyświetlane jednostki
>1000 (µs)	µs
$10^6 > x > 10^3$ (ms)	ms

$6*10^7>x>10^6$(s)	s, ms
$3,6*10^9>x>6*10^7$(min)	min, s
$8,64*10^{10}>x>3,6*10^9$(h)	h, m, s
$8,64*10^{10}>x$(d)	d, h, m, s

Argumenty:

- duration – liczba mikrosekund

u32 findLastLesserinSorted(u64 *list, u32 first, u32 last, u64 what)

Używa wyszukiwania binarnego, by szybko znajdować liczby w liście.

Argumenty:

- list – wskaźnik do listy

- first i last – początek i koniec listy

- what – co znaleźć

Wartość zwracana: indeks do tablicy elementu

Testy

Nazwa	Opis	SWT
Obecność wyjątku AddressSanitizer	Gdyby taki wyjątek został rzucony przez ten program, to należy naprawić zarządzanie pamięcią w programie. Żeby móc wykrywać take błędy, należy przy kompilacji włączyć flagę *-fsanitize=address*	F
Wydajność programu przy N = 10^{12}	Test wydajności programu przy wyznaczaniu $\pi(10^{12})$. Celem jest uzyskanie min. 3min 20s czasu wykonywania programu.	F
Brak wycieków danych	Czy tablice dynamiczne są na bieżąco usuwane, by nie zajmować niepotrzebnie pamięci?	F

Benchmark

Wszystkie testy zostały przeprowadzone na sprzęcie z następującymi podzespołami:

Procesor	Intel Core i5-7400
Taktowanie procesora	3.00 GhZ / 3.50 GHz (w trybie turbo)
CPU Overclock	Nie
RAM Overclock	Nie
Chipset płyty głównej	H110
Pamięć RAM	8GB DDR4-2400 SK Hynix
System operacyjny	Omarchy Linux Quattro, Linux 7.2.11-arch1.1

Należy wspomnieć, że wyniki te dotyczą wersji kompilacji *Release* bez możliwości skutecznego debugowania

Wyniki wyglądają następująco:

Zakres do	Spodziewana liczba liczb pierwszych	Czas (s)
10^7	664,579	0.002-0.005
10^8	5,761,455	0.023-0.03
10^9	50,847,534	0.232-0.239
10^{10}	455,052,511	2.5-2.744
10^{11}	4,118,054,813	27.757

Wymagania aplikacji

Nazwa	Ważność	Stan
Program oferuje przyjazny dla uzytkownika zestaw opcji.	Niezbędna	Zawiera tylko 2 opcje: włączenie wielowątkowości i podanie końcowej liczby
Program obsługuje wielowątkowość	Opcjonalna	Tak, poprzez OpenMP
Program zwraca dla dowolnej liczby n > 7 poprawny wynik	Niezbędna	Tak, ale zwracany wynik może mylić się o +1, a dla liczby 8 program wyrzuca wyjątek SIGSEGV
Program pozwala na zapis do pliku liczb pierwszych	Opcjonalana	Brak
Program potrafi generować liczby pierwsze od podstaw – bez użycia słownika	Opcjonalna+	Brak

Definicje Testów

Testy jednostkowe to testy, w których testujemy daną funkcję osobno, w celu zweryfikowania, czy funkcja działa poprawnie dla dowolnych argumentów z oferowanych typów argumentów (np. w celu sprawdzenia działania funkcji dla szczególnych przypadków, jak wysoki wykładnik w funkcji potęgowania, czy reakcja komputera na dzielenie przez 0)

xUnit to otwartoźródłowy program do testowania jednostkowego, która jest włączona w .NET Foundation (czyli jest deployowana wraz z .NET runtimem).

Testy integracyjne to inaczej testy sprawdzające, czy komponenty składające się na program/aplikację prowadzą do poprawnych wyników. Na przykład można przetestować, czy zewnętrzne API (jak backend, komercyjne z danymi) zwraca pożądane dane w pożądanym formacie (np. csv, json, plaintext), zamiast błędów. Można przecież przypadkowo wysłać zapytanie, na coś, co na serwerze nie istnieje.

Testy systemowe to testy sprawdzające całość aplikacji, jego integracji z API i pozostałymi komponentami. Stosuje się je w celu wykrycia luk w oprogramowaniu, sprawdzenia wydajności aplikacji, czy podatność na błędy czy niespójności z zewnętrznymi API.

Na przykład można zastosować takie testy w przypadku tworzenia silnika gier, by móc sprawdzić pod koniec tworzenia programu spójność UI, funkcji matematycznych, integracji z popularnymi programami typu edytor grafiki 3D, czy poprawna obsługa typów danych grafiki 2D/3D, muzyki, audio-wideo itd.

Testy akceptacyjne to takie, które sprawdzają, czy aplikacja działa zgodnie z oczekiwaniami klienta/deploymentu.

Powiedzmy, że mamy na zlecenie wykonać aplikację do zarządzania finansami, która może współpracować z Profilem Zaufanym. Klient może sobie życzyć zainplementowanie funkcjonalności, takie jak: integracja z kontem bankowym, możliwość tworzenia osobnych *skrytek* na oszczędności/cele finansowe, jak zakup konsoli do gier, Iphone'a, gitary, itd., narzędzia przewidywania stanu konta.

Po wykonaniu powyższych testów, właśnie testy akceptacyjne wchodzą w grę. To tutaj klient ma okazję przetestować aplikację, by sprawdzić, czy jego wymagania zostały spełnione.

Do nich zaliczają się testy czarnej skrzynki. Polegają na sprawdzaniu bez wglądu w strukturę, czy program spełnia swoją rolę i zwraca poprawne wyniki

Te testy przechodzą w:

- Testy alfa – przeprowadzane przez dowolny zewnętrzny zespół testerów.

- Testy beta – przeprowadzane już przez zewnętrznych użytkowników.